# Session 1 Storybook

Classroom: Agentische Entwicklung mit Claude Code, Mastra & CopilotKit, Session 1

# What we do today



This is the live-coding script for the hands-on part of session 1. Each step gives you a **Goal**, the **Prompt** to hand Claude Code, **Teaching points** to narrate while the agent works, and a **Verify** checklist. Every prompt here ran end-to-end under `claude -p --model claude-opus-5`, and the finished reference app lives in `session-01/ai-tutor`.

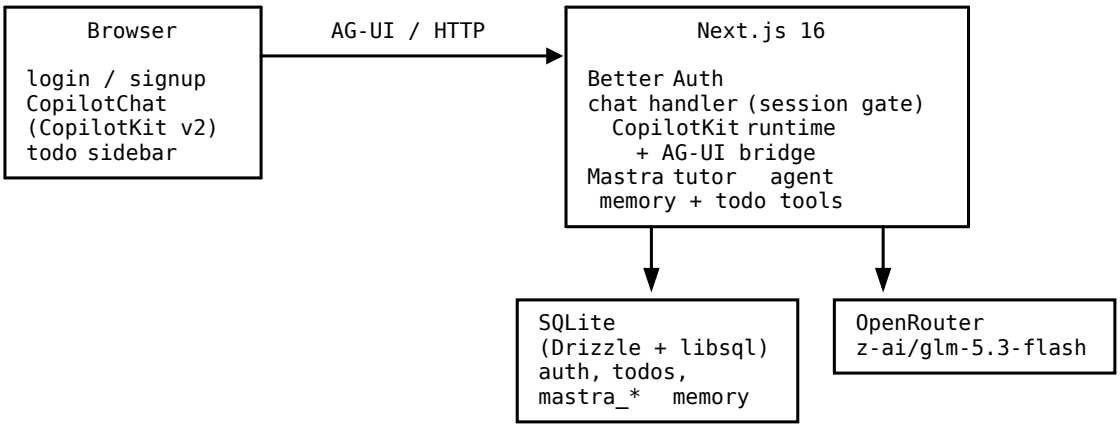
The prompts are short and natural on purpose. A frontier model doesn't need step-by-step instructions. Give it the *outcome* and the *constraints that matter*, then point it at *current docs*, and it fills in the rest. That's the prompting style we teach all day.

## What we build today

**ai-tutor** is a minimal AI tutoring chat, the day-1 slice of the app we grow over the five sessions.

- **Next.js 16** is the web app: App Router, TypeScript, Tailwind v4, Biome.
- **Better Auth** handles email and password sign-up and sign-in, so every chat belongs to one user.
- **SQLite** in a single file, with the path from `.env` and all access through **Drizzle**, holds the app tables, the auth tables, and the agent memory.
- **Mastra** runs one `tutor` agent with a **hardcoded system prompt** and memory per user.
- **OpenRouter** supplies the model behind that agent, `z-ai/glm-5.3-flash`.
- **CopilotKit + AG-UI** is the chat UI, wired to the Mastra agent.
- **Tool calling** gives the student a todo list they manage *through the agent*.
- **Vitest and Playwright** cover integration tests (with Better Auth's test utils) and e2e.

How the pieces fit, with the svgbob source in `images/architecture.bob` and the render in `images/architecture.svg`:




Later sessions add the CLI, YAML-defined activities, more activity modules, usage metering, CI/CD, and deployment. None of that today.

## What we *teach* today (the meta-level)

The app is the vehicle. The lesson is **how to drive a coding agent**.

1. **Prompting.** Outcome, constraints, and acceptance criteria, instead of micromanagement.
2. **Grounding in current docs.** Training data goes stale for fast-moving libraries, and the fix is *not* pasting docs into the prompt or babysitting the model. Give the agent **sources it can pull from itself** and it manages its own knowledge. That's the core message of the day: with llms.txt, node_modules docs, skills, and context7 you point a modern LLM at the truth and review the result instead of micro-managing it. Each mechanism reaches a different kind of source.
    - llms.txt gives you vendor-curated, agent-readable doc indexes, the way Better Auth and Drizzle publish them.
    - **Docs in** node_modules**.** Next.js 16 ships its documentation *inside the package*, exact for the installed version.
    - **Skills** are installable playbooks from skills.sh, and not only for tech. We install tech skills (find-docs, mastra, CopilotKit's), a process skill (grilling), a meta-skill (skill-creator), and a design skill (frontend-design), so expertise, workflow, and taste all arrive as packages.
3. AGENTS.md **and** CLAUDE.md **are agent-maintained memory.** Short and current, updated by the agent itself as part of every change, never a dumping ground.
4. **Verification belongs in the prompt.** The agent proves its work by running the tests and the build. You still review the diff like a pull request.

### Step 0: prerequisites (before the session)

- Node.js **24 or newer**, and Claude Code installed (check `claude --version`).
- An **OpenRouter API key**. We use `z-ai/glm-5.3-flash`, which is cheap.
- A terminal and an editor. Nothing else, because SQLite is a local file and there are no cloud accounts.

### Step 1: scaffold the app (no agent)

**Goal** a fresh Next.js app with TypeScript, Tailwind, and Biome, plus our `.env`.

```
npx create-next-app@latest ai-tutor \
  --ts --tailwind --biome --app --no-src-dir --turbopack --import-alias "@/*"
cd ai-tutor
git add -A && git commit -m "scaffold"

cat > .env <<'EOF'
# SQLite database file (Drizzle, BetterAuth, and Mastra memory all use it)
DATABASE_URL=file:./data/app.db
# OpenRouter — the LLM behind the Mastra agent
OPENROUTER_API_KEY=sk-or-v1-...
# BetterAuth (secret: openssl rand -base64 32)
BETTER_AUTH_SECRET=...
BETTER_AUTH_URL=http://localhost:3000
EOF
```

#### TEACHING POINTS

- **Don't use an agent for deterministic work.** Scaffolding is a solved, scripted problem, and the generator is faster and cheaper. It also produces the same result every time. Agents are for work that needs judgment.
- `create-next-app` now offers **Biome** instead of ESLint, one fast tool that lints and formats.
- Look at what the scaffold created. `AGENTS.md` is the memory file, and `CLAUDE.md` is a one-line `@AGENTS.md` include, so there is one memory file and any agent can read it.
- `AGENTS.md` already carries Next's `nextjs-agent-rules` block, which opens with "This is NOT the Next.js you know" and points agents at `node_modules/next/dist/docs/`. The framework ships current docs inside the package because models keep writing outdated Next.js code from training data. Run `ls node_modules/next/dist/docs/` to show it, and note that `next dev` puts the block back if you delete it.
- The scaffold git-ignores `.env`, so secrets never reach the repo. They never reach the browser either, because only server-side modules read them.
- From now on, make **one commit per step**, so each diff is small enough to review what the agent did.

**VERIFY** `npm run dev` shows the welcome page, and `AGENTS.md` exists.

## Step 2: a short AGENTS.md that maintains itself

**Goal** turn AGENTS.md into concise, *self-maintaining* project memory.

Start claude. You can show /init first, which analyzes the repo and writes the memory file. Either way, the prompt is the same.

**Prompt 2.1**

> Make AGENTS.md a concise memory for future agent sessions — a map, not a manual. One line on what this app is, the commands, and per area a pointer to the key files plus only what an agent would get wrong *even after reading those files*: hard-won gotchas, cross-file invariants, never-do rules. Anything discoverable by reading the file a bullet points to doesn't belong here. One sentence per bullet. Keep the nextjs-agent-rules block. End with a maintenance rule addressed to you, the agent: update this file in the same change set whenever a change invalidates a line or teaches a costly lesson; prefer deleting over adding, pointers over prose; one sentence per bullet, current state only, no history.

### TEACHING POINTS

- The memory file is **prepended to every session**, so it costs context tokens every time. Short and dense beats complete. If ls or package.json already answers the question, it doesn't belong here.
- **A map, not a manual.** Every bullet faces one test: would an agent still get this wrong *after* reading the files the bullet points to? If yes, keep it. Version pins, fail-closed rules, and lessons that cost someone a debugging session all pass. If no, the pointer alone does the job. Narrated implementation is the first thing to go stale.
- **Constrain shape, not count.** Say "keep it under 60 lines" and the agent meets the cap by packing four sentences into every bullet. Metrics get gamed. "One sentence per bullet" can't be, and it keeps working as the file grows over the later steps. Worth a 60-second aside on specification writing.
- The maintenance rule is the trick. From now on **nobody edits this file by hand**, and the agent keeps it current as a side effect of normal work. Watch for AGENTS.md in the diff of every later step.
- /init is a fine starting point on an *existing* codebase. On a fresh scaffold there's little to analyze, so saying what you want is quicker. Either way, the trimming instinct is the skill.

**VERIFY** AGENTS.md reads as pointers plus one-sentence gotchas, and it carries both the maintenance rule and the nextjs-agent-rules block. Commit.

## Step 3: install skills

**Goal** current expertise for the fast-moving parts of the stack.

```
# Tech skills — current knowledge for fast-moving libraries
npx skills add upstash/context7 --skill find-docs --agent claude-code -y
npx skills add mastra-ai/skills --skill mastra --agent claude-code -y
npx skills add CopilotKit/CopilotKit --agent claude-code -y \
  --skill copilotkit-setup --skill copilotkit-agui \
```

```
  --skill copilotkit-develop --skill copilotkit-debug

# Process skill — how we work, not what we build
npx skills add mattpocock/skills --skill grilling --agent claude-code -y

# Meta-skill + design skill
npx skills add anthropics/skills --skill skill-creator --skill frontend-design \
  --agent claude-code -y
```

(npx skills add <owner/repo> -l lists what a repository offers before installing.)

**TEACHING POINTS**

- **A skill is installable expertise**, a Markdown playbook plus a when-to-use trigger, loaded on demand. It costs no context until it fires. skills.sh is the registry and npx skills find <query> searches it. skills-lock.json pins what you installed, the way a package manager does.
- Skills come in **different kinds**. Walk through the four we just installed.
  - **Tech skills** teach the agent *current technology*. find-docs from Context7 is the universal fallback, looking up current docs for any library through the ctx7 CLI instead of trusting training data. The mastra and CopilotKit skills come from the vendors themselves, because framework authors now ship the playbook for wiring their own product.
  - **Process skills** encode *how development runs*, meaning requirements, reviews, and workflow. grilling by Matt Pocock turns the agent into a relentless interviewer that stress-tests a plan or a design *before* anyone writes code. Live beat: say "grill me about the tutor app plan" and let it ask one round of questions. Sixty seconds makes the point that prompting runs in both directions.
  - **Meta-skills** are skills about skills. skill-creator from Anthropic teaches the agent to *author* well-formed skills, which is the path from "we explained this convention three times" to "it's a skill now". Finding skills is covered by npx skills find, and a find-skills skill exists too.
  - **Design skills** carry *taste*. frontend-design from Anthropic pushes UI work away from templated defaults toward deliberate typography and a chosen aesthetic direction. It fires by itself when later steps touch the UI.
- Commit the skills and the lock file, and every teammate's agent gets the same expertise, the process and the taste included, not just the tech.

> **VERIFY** the skills sit under .claude/skills/ (or .agents/skills/) and skills-lock.json exists. A fresh claude session lists them in /context.

### Step 4: the Vitest and Playwright test harness

**Goal** testing infrastructure *before* features, so every later step has to prove itself.

**Prompt 4.1**

Set up our test harness: Vitest for unit/integration tests (`npm test`) and Playwright for e2e (`npm run test:e2e`, Chromium only, starting its own dev server on a spare port). This Next.js version may differ from what you know — check its testing guides in node_modules/next/dist/docs first. Add one real smoke test for each, make everything pass including `npx biome check .`, and update AGENTS.md per its rule.

**TEACHING POINTS**

- Anatomy of a good prompt. It names the *outcome* (harness plus scripts), the *constraints* (Chromium, its own server, a spare port), the *docs pointer* (node_modules), the *verification*, and the *memory* update. Five lines. Configs, plugins, and ports are the agent's job.
- This is the rhythm of the day. You prompt, the agent works for a few minutes, then you review the diff like a pull request. Narrate the tool calls as they scroll by, the file reads and doc lookups and test runs, because *that* is what makes it an agent rather than a chatbot.
- Check the AGENTS.md diff. The test commands appeared and nothing bloated. The rule works.
- Findings from the validation run, likely to recur live. The agent noticed that the bundled Vitest guide was *itself* slightly stale, naming a deprecated plugin, and it followed the runtime warning instead. Docs ground the model, they don't replace its judgment. It also excluded `.claude/skills/**` from Biome, because those files are vendored and any reformatting would be lost on the next reinstall, and it *reported* that decision rather than burying it. Reading the agent's summary is part of the review.

**VERIFY** `npm test` and `npm run test:e2e` come back green. Review the diff, then commit.

### Step 5: Drizzle and SQLite, with the path from `.env`

**Goal** the persistence foundation, Drizzle on a local SQLite file whose path comes from `.env`.

**Prompt 5.1**

Add persistence: Drizzle ORM on SQLite via @libsql/client, using DATABASE_URL from .env (already set to `file:./data/app.db`; keep `data/` git-ignored). One server-only module `lib/db.ts` exports the drizzle instance — nothing else opens the database. Set up drizzle-kit migrations (`npm run db:generate` / `db:migrate`) and a first table `todos` (id, userId, title, done, createdAt) — auth and agent tools will build on this later. Add a Vitest test that migrates a temporary database file and does a real insert/select. Drizzle's API has changed a lot — read https://orm.drizzle.team/llms.txt and follow the relevant links before coding. Update AGENTS.md per its rule.

**TEACHING POINTS**

- `llms.txt` (llmstxt.org) is an agent-readable index that vendors publish at a stable URL. The agent fetches it and follows the right sub-pages. One URL in the prompt beats twenty lines of pasted docs, and it's current every time you run it. Watch it happen in the tool stream.
- **One seam for the database.** Demanding `lib/db.ts` now pays off in steps 6 through 8, when auth and agent memory land in the same database file, because exactly one module interprets `DATABASE_URL`.

- Tests run against a temp file while dev runs against `data/app.db`, so SQLite hands you isolation for free. That's why it's the classroom database. We pick libsql over better-sqlite3 because the same `file:` URL works for Drizzle and, later, for Mastra's storage. One driver everywhere.

> **VERIFY** `npm test` is green, `npm run db:migrate` creates `data/app.db`, and `git status` shows nothing from `data/`. Commit.

### Step 6: authentication with Better Auth

**Goal** sign-up, sign-in, and sign-out, with the app behind a login and the flows tested using Better Auth's official test utils.

**Prompt 6.1**

> Add authentication with Better Auth: email + password only. Use its Drizzle adapter on our lib/db.ts, generate the auth schema with their CLI, and apply it through our normal migration flow (secret and URL are already in .env). Build /signup and /login pages with Tailwind — factor shared form styling into components/ui/ rather than duplicating class strings — and gate the app: / requires a session (checked server-side) and shows the user's name plus sign-out. Tests: Vitest integration tests using Better Auth's official test-utils plugin on a temporary database (sign-up works, correct password signs in, wrong password rejected) plus one Playwright e2e of the real flow. Better Auth is newer than your training data — start at https://better-auth.com/llms.txt and follow its Next.js, Drizzle adapter, email-password, and test-utils pages. Update AGENTS.md per its rule.

#### TEACHING POINTS

- Still outcomes, not steps. The library-specific "how", meaning adapter config, CLI schema generation, and cookie handling, goes to **the vendor's llms.txt**. Without that pointer you get a plausible 2025-vintage Better Auth API that no longer exists.
- **The auth schema is generated, not hand-written**, and it still flows through our one migration path. Generators inside an agent workflow are fine, and the agent runs them itself.
- The `components/ui/` requirement plants the **Tailwind discipline** for the whole course: shared components, no copy-pasted class recipes. Demanding it now costs less than refactoring later.
- **The session check runs server-side.** This is the first security beat of the course. A client check is there for the user experience, and the server check is the actual gate. The same point comes back in steps 7 and 8.
- The **test-utils plugin** (`testUtils` from `better-auth/plugins`, inside a test-only auth instance) runs real auth flows in Vitest without HTTP or a browser. Fast tests cover the logic, and one Playwright test covers the real wiring.

> **VERIFY** the e2e suite is green. By hand, sign up, delete the cookies, and confirm the gate redirects to /login. In the diff, confirm the session check lives server-side. Commit.

### Step 7: the agent, Mastra and OpenRouter behind CopilotKit

**Goal** the heart of the app. Signed-in users chat through CopilotKit with a Mastra tutor agent that has a hardcoded system prompt and per-user memory in our SQLite file.

**Prompt 7.1**

> Now the heart of the app: chat with an AI tutor. Use your mastra and copilotkit skills — don't wire this from memory. One Mastra agent `tutor` with a hardcoded system prompt you write (a patient, british butler maintaining a todo list for the user; rejects any request not related to the todo list), model `z-ai/glm-5.3-flash` via OpenRouter (OPENROUTER_API_KEY in .env, server-only), and Mastra memory persisted in our SQLite file so conversations survive restarts. Serve the agent to CopilotKit via the AG-UI integration and make / the chat page (keep the header with sign-out; style with Tailwind, reuse components/ui/). The chat route must reject unauthenticated requests, and memory must be scoped by the Better Auth user id — one thread per user for now; user A must never see user B's conversation. Add a Vitest test for the auth gate, keep everything green (tests, build, biome), and update AGENTS.md — the agent, the chat route, and the memory scoping are things a future session must know.

**TEACHING POINTS**

- **Skills carry the integration knowledge.** Mastra, AG-UI, and CopilotKit form a three-package handshake that changes fast, and the vendors' skills encode today's correct wiring. Writing "don't wire this from memory" turns grounding into an explicit instruction. For a stack that moves this fast, say it.
- **The system prompt is product, not config.** Today it's hardcoded on purpose. In a later session it becomes data, per-activity YAML like the big prototype uses. Shipping the simple verified version first *is* agentic development.
- **Memory scoping is authorization.** The resource id comes from the *server-side session*, never from the client payload. That's where multi-tenant chat privacy actually lives.
- Secrets stay server-side. The browser talks to our route, and only the server talks to OpenRouter.
- Live demo: chat with it ("explain Big-O like I'm 12"), restart the dev server, and reload. The conversation survives, because the memory sits in SQLite. Open a second browser or an incognito window, sign up as a second user, and that chat is separate and empty.
- Findings from the validation run. Opus used Mastra's model router, where the string `openrouter/z-ai/glm-5.3-flash` picks the provider and the model and reads the key from the environment. It pinned `@ag-ui/client` and `@ag-ui/core` to 0.0.57 and `@ag-ui/mastra` to 1.1.1, because CopilotKit 1.69.x pins them exactly and two copies produce type errors. It also found on its own that CopilotKit's runner keeps a process-global thread cache keyed by thread id alone, and it added a guard that rejects any request naming a thread other than the caller's. Discuss that one. The *model* found a multi-tenancy hole the prompt only hinted at with "user A must never see user B's conversation", and an outcome-oriented prompt is what buys you that.

**VERIFY** a manual round-trip works, the conversation survives a restart, two users stay isolated, and `npm run build` is green. Commit.

### Step 8: tool calling, the agent manages your todos

**Goal** the "aha" of agentic apps, where the LLM *acts*. It reads and writes the `todos` table through typed tools, and the UI shows the result right away.

**Prompt 8.1**

> Give the tutor tools to manage the signed-in student's todo list, backed by our todos table: listTodos, addTodo, setTodoDone. The user id comes from the server-side session through the agent's runtime context — never from the model or the client. Extend the system prompt so the tutor offers to capture action items and mark them done. Add a slim todos sidebar next to the chat (read-only — the agent is the write path) that refreshes when the agent changes something. Tests: Vitest for the tool executors on a temporary database (especially per-user isolation), plus one Playwright e2e that is NOT part of the default suite (`npm run test:e2e:llm`, it costs LLM calls): sign up, ask the agent to add "buy milk", assert it appears in the sidebar. Use the mastra skill for the current tools API. Update AGENTS.md per its rule.

**TEACHING POINTS**

- **Tools are the contract between the LLM and our system.** The Zod schema plus the description is the API documentation the model reads, so writing them well *is* prompt engineering.
- **Identity injection** runs from the session into the runtime context and on into the tool executor. The model never sees the user id and never chooses it. Same point as steps 6 and 7: the model is untrusted input, and authorization lives in our code.
- **LLM tests are quarantined.** They're non-deterministic and slow, and they cost real money on every run, so they get their own npm script and stay out of the default suite. In the big prototype this grows into a whole `@live-llm` taxonomy.
- Live demo: say "I need to read chapter 3 and do exercise 5 for tomorrow" and the tutor offers to capture the todos. The sidebar updates. Say "finished the reading" and the checkmark flips.
- Findings from the validation run. The tools declare `requestContextSchema`, so a tool invoked without a user id *errors* instead of falling back to a default, which is failing closed. The LLM e2e passed on the second try, because Enter doesn't submit CopilotKit's composer and the spec has to click the send button. The quarantine landed as a separate Playwright config, `playwright.llm.config.ts`, rather than a grep tag, and that structure holds up better than a naming convention.

**VERIFY** the live demo works, the unit tests prove per-user isolation, and the default e2e suite stays free of LLM calls. Commit.

## Step 9: quality pass and wrap-up

**Goal** leave the repo the way every session should end, clean and tested, with a README that matches it.

**Prompt 9.1**

> Quality pass over the whole repo. Fan out subagents in parallel, on the cheapest model that's up to each job: a Sonnet agent sweeps for duplicated Tailwind styling and consolidates it into components/ui, a second Sonnet agent writes a concise README (what the app is, stack, setup incl. env vars, scripts, short architecture overview — current state only), and an Opus agent reviews

> the auth, chat, and tool code with fresh eyes for security problems (session checks, user scoping, secrets) and reports findings without editing. Then integrate: fix anything the review found, run `npx biome check --write .`, make the full suite green (tests, e2e, build), and check AGENTS.md against its own rule — current, concise, nothing stale.

## TEACHING POINTS

- **Subagents.** Claude Code can spawn parallel agents, each with its own fresh context. You see both reasons to use them here. *Parallelism* handles disjoint chores like the styling sweep and the README. *Fresh eyes* handle the review, because the reviewer subagent never saw the implementation history and reads the code like an outside auditor instead of trusting its own memory of writing it. Note the constraint "reports findings without editing". Parallel agents get disjoint write areas, and reviewers stay read-only.
- **Model tiering** is the third reason for subagents, and it's about *cost*. The coordinator session (Fable in the live demo) is the expensive part, because it holds the whole history and makes the judgment calls. Each subagent picks its own model, so match the tier to the task. Mechanical chores like the styling sweep and the README run fine on Sonnet; a repo-wide sweep is the kind of work agents do best and humans keep postponing. The security review is judgment work, where you're paying for what you *didn't* think to ask, so it stays on Opus. Rule of thumb: if you could write the task as a checklist, tier down, and never tier down the reviewer.
- In the validation run the fresh-eyes reviewer earned its keep with five findings. Two were real medium-severity holes: a route-guard gap that exposed every user's thread list, and an unguarded clear-all endpoint. It verified each one against the library source before anything got fixed, and it flagged an unthrottled login path as well. The implementing agent had *tested* its guard, while the reviewer attacked the shape of it. That difference is the demo.
- The README is for humans arriving at the repo, and AGENTS.md is operating instructions for agents. Both hold current state only.

> **VERIFY** the full suite is green and the README reads well. Commit.

## Step 10: a project design skill, from brand to product

**Goal** the skill kinds from step 3 working together. The meta-skill and the design skill combine to mint a skill of our own, and then two sentences restyle the whole app.

**Prompt 10.1**

> Study heise.de — colors, typography, the overall design language. Then use the skill-creator and frontend-design skills to distill a project-specific design skill for this app. One thing the skill must make clear: unlike heise.de, this app is not a news page but a chat bot with data-heavy pages, and the design has to reflect that — say what carries over from the brand and what doesn't. Just create the skill, don't restyle anything yet.

**Prompt 10.2**

> Apply the ai-tutor-design skill to the existing UI. Make the full suite green afterwards and keep AGENTS.md current.

**Prompt 10.3**

> The restyle uncovered two traps in CopilotKit's token layer — AGENTS.md records them, but the ai-tutor-design skill's recipe still gets them wrong. Fold the fixes back into the skill. Don't change the app.

### TEACHING POINTS

- **Skills compose.** skill-creator supplies the form, frontend-design supplies the taste, and the agent's own research supplies the project truth. The output is a *new* skill in .claude/skills/ai-tutor-design/ that came from no registry. It's your own design system, packaged as installable expertise.
- **The agent measured instead of guessing.** In the validation run it pulled heise.de's actual stylesheets: the brand tokens #0056A4 ink blue and #E8EDF0 page gray, Source Sans with no second family, *eight* border-radius declarations across 152 KB of CSS, and none for desktop shadows. The evidence lands in references/heise-reference.md, so anyone can check a claim in the skill instead of re-deriving it. Same discipline as llms.txt, pointed at a brand instead of an API.
- **Translation, not copy.** The constraint in the prompt does real work. The skill's core is two tables. One holds what carries over, meaning the palette, square corners, blue as the actionable color, and hairlines instead of shadows. The other holds what dies here, meaning teaser grids, hero images, and kickers, summed up in the skill's own line: "if you find yourself building a card grid, you are designing the wrong app". The best find is heise's signature vertical marker bar, which becomes the tutor's turn marker in the stream and replaces chat bubbles entirely.
- **Prompt 10.2 is two sentences.** The skill carries all the detail, and its description tells the agent when to fire, so every future "add a settings page" gets the design system for free. That's the day's message in miniature: build reusable context once, then stop micro-managing.
- Findings from the validation run. Applying the skill *tested* the skill. The restyle agent hit two bugs in the recipe the skill ships for CopilotKit's token layer (a custom-property

rule that shadows the very variables it reads, and an override block that loses on specificity to CopilotKit's unlayered .dark selectors), worked around both, and recorded them in AGENTS.md rather than burying them. It also went one deliberate step past a pure restyle: a turn that only calls a tool renders no text, which left the tutor's marker rule floating as an orphan tick, so the tools now narrate themselves ("Added to your action items"). Both calls surfaced in the agent's summary, and reading the summary is where you catch them.

- **A skill is code, and its first application is its first test.** That is what prompt 10.3 is for. The fix belongs in the skill, not in a workaround the next agent has to rediscover.
- Close the loop with git log --oneline. A dozen small steps, each one reviewed and tested. Agentic development is that conversation of scoped, verified moves rather than one giant prompt.

> **VERIFY** the skill exists with its references/ directory, the app is restyled and visibly *not* default-Tailwind, the skill's recipe matches the shipped CSS, and the full suite is green. One commit per prompt.

## Appendix A: running this storybook headless

Every prompt ran non-interactively, one step per invocation, with fresh context each time:

```
cd session-01/ai-tutor
claude --model claude-opus-5 --dangerously-skip-permissions -p "<prompt>"
```

In the live session, use the interactive TUI. The audience should see tool calls, doc fetches, diffs, and test runs scroll by. That stream *is* the content.

The Sonnet and Opus tier hints in prompt 9.1 came after the validation run, which used default-model subagents under an Opus coordinator. The hints change what the subagents cost, not what the prompt asks for.

One gotcha: `create-next-app` only runs `git init` when the new folder is *not* already inside a git repository. In a nested folder like this classroom repo, run `git init -b main` inside `ai-tutor` yourself before the scaffold commit.

## Appendix B: live-demo insurance

- One commit per step means you can `git reset --hard` and retry, or fast-forward from the reference repo in `session-01/ai-tutor`.
- Step 7 is the most fragile, because of the three-package handshake. If versions drift on workshop day, pin the reference repo's `package.json` versions in the prompt. The combination that worked: `@copilotkit/react-core` and `runtime` 1.69.3, `@ag-ui/mastra` 1.1.1, `@ag-ui/client` and `@ag-ui/core` pinned to 0.0.57, `@mastra/core` 1.63.x with `@mastra/memory` and `@mastra/libsql`, `drizzle-orm` 1.0.0-rc.4, `better-auth` (see the reference repo), and Next 16.3.3.
- The LLM e2e flaking live is *content* rather than a failure, and it's exactly why we quarantined it.